

CS3002E Compiler Design

Monsoon 2025

Lecture #7

Parsing: Introduction

Department of Computer Science & Engineering
NIT Calicut

Compiler - Analysis

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis

Lexical Analysis

- Input: Program (as a character stream), and the specification of tokens.
- Output: Stream of tokens (one token corresponding to each *lexeme*).

Syntax Analysis or Parsing

- Input: Token Stream, Syntax Specification of the programming language in the form of Context Free Grammar(CFG).
 - Grammar rules describe the syntactic structure of well-formed programs
- Checks if the sequence of token names can be generated by the grammar, by simulating the steps in building a parse tree for the input.
- Output: Parse Tree (or some Intermediate Representation of the program), if parsing is successful.

Syntax Descriptions - origins

- Noam Chomsky (1956): introduced CFG formalism as part of study on natural languages.
- John Backus (1959): specification of the FORTRAN programming language.
- Peter Naur (1960): specification of Algol 60.

Backus-Naur Form (BNF)

$$\begin{aligned} \langle \text{expression} \rangle ::= & \langle \text{expression} \rangle + \langle \text{expression} \rangle \\ & | \langle \text{id} \rangle \\ & | \langle \text{num} \rangle \end{aligned}$$

Parsing

Given Grammar G and a string w test if $w \in L(G)$ ¹

¹ $L(G)$ - Language generated by G

Parser- types

- Universal
 - Parse any grammar - e.g. CYK Algorithm - Cocke, Younger and Kasami - $O(n^3)$
- Compilers commonly use
 - Top-Down Parsing - parse tree built from top(root) to bottom(leaves)
 - Bottom-Up Parsing - parse tree building starts from the leaves

Parsing

Input: Stream of Tokens and a CFG describing the language.

- Checks if the stream of token names can be generated by the grammar.
- Generates a parse tree (or some Intermediate Representation of the program).
- Error reporting/recovery.
- Creates / updates Symbol Table entries.

Context Free Grammar

$$G = (N, T, P, S)$$

N : a set of *Nonterminals* (also known as *Variables*). Each nonterminal denotes a set of strings.

T : a set of *terminal* symbols, the basic symbols from which strings are formed (names of tokens are treated as terminals)

P : a set of *productions* (also known as grammar rules)

S : the *start* symbol, a nonterminal.

Normally, the grammar is specified by listing the productions, with the productions for the start symbol listed first.

CFG: Production

- Each production is of the form $A \rightarrow \alpha$ where A is a nonterminal, and α a string of zero or more terminals and non terminals.
- In a production $A \rightarrow \alpha$, A is known as the **head** of the production and α , the **body** of the production.

Context Free Grammar

$$S \rightarrow \text{if } (E) S \text{ else } S \mid \text{id} = E$$
$$E \rightarrow E + E \mid \text{id}$$
$$N = ?$$
$$T = ?$$

Context Free Grammar

$$S \rightarrow \text{if } (E) S \text{ else } S \mid \text{id} = E$$
$$E \rightarrow E + E \mid \text{id}$$
$$N = \{S, E\}$$
$$T = \{\text{if}, (,), \text{else}, =, +, \text{id}\}$$

S is the start symbol

CFG - alternate notation

$stmt \rightarrow \mathbf{if} (expression) stmt \mathbf{else} stmt \mid \mathbf{id} = expression$

$expression \rightarrow expression + expression \mid \mathbf{id}$

$N = \{stmt, expression\}$

$stmt$ is the start symbol

CFG: Different notations

Notation 1: $expression \rightarrow expression + expression$
 | **id**
 | **num**

Notation 2: $E \rightarrow E + E$
 | **id**
 | **num**

Derivation

- Derivation of a string begins with the start symbol.
- A step of derivation replaces a nonterminal with the body of a production for that nonterminal.
- The strings of terminals that can be derived from the start symbol form the language defined by the grammar.

Derivation: example

string: **id = id + id**

$S \Rightarrow id = E \Rightarrow id = E + E \Rightarrow id = id + E \Rightarrow id = id + id$

Derivation

$$S \Rightarrow id = E \Rightarrow id = E + E \Rightarrow id = id + E \Rightarrow id = id + id$$

Derivation step: replaces a nonterminal A by a string of grammar symbols α , where $A \rightarrow \alpha$ is a production ²

² $\alpha, \beta, \gamma \dots$ denote strings of grammar symbols

Derivation

$$S \Rightarrow id = E \Rightarrow id = E + E \Rightarrow id = id + E \Rightarrow id = id + id$$

$$S \Rightarrow^* id = id + id$$

$\Rightarrow^*$: derives in zero or more steps

Sentential Form

If $S \xRightarrow{*} \alpha$, we say α is a *sentential form*

A sentential form may contain both terminals and nonterminals, and may be empty.

Sentence

A **sentence** is a sentential form with no nonterminals.

Language

$L(G)$, the language generated by grammar G is set of sentences derivable from the start symbol of G .

Language

A string of terminals $w \in L(G)$ if and only if $S \xRightarrow{*} w$ ³

³ $u, v, w \dots$ denote strings of terminals

Derivations

$$S \Rightarrow id_1 = E \Rightarrow id_1 = E + E \Rightarrow id_1 = id_2 + id_3$$

$$S \Rightarrow id_1 = E \Rightarrow id_1 = E + E \Rightarrow id_1 = E + id_3 \Rightarrow id_1 = id_2 + id_3$$

Derivations

Leftmost derivation

$$S \Rightarrow id_1 = E \Rightarrow id_1 = E + E \Rightarrow id_1 = id_2 + id_3$$

Rightmost derivation

$$S \Rightarrow id_1 = E \Rightarrow id_1 = E + E \Rightarrow id_1 = E + id_3 \Rightarrow id_1 = id_2 + id_3$$

Derivations

Leftmost derivation: Each step replaces the leftmost nonterminal in the sentential form

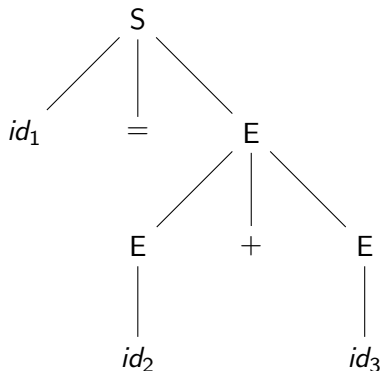
Rightmost derivation: Each step replaces the rightmost nonterminal in the sentential form.

Parse Tree

A Parse tree is a graphical representation of a derivation.

- The root of the tree is labeled by the start symbol.
- Each leaf node is labeled by a terminal or ϵ .
- Each interior node is labeled by a nonterminal.
- If a step in a derivation replaces A by XYZ , the parse tree has a corresponding internal node A with children labeled X , Y , Z , from left to right.

Parse Tree: example



Derivation:

$$S \Rightarrow id_1 = E \Rightarrow id_1 = E + E \Rightarrow id_1 = id_2 + E \Rightarrow id_1 = id_2 + id_3$$

The yield of a parse tree

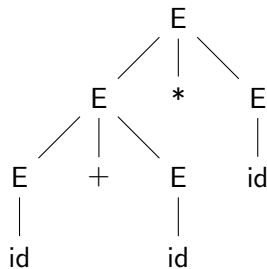
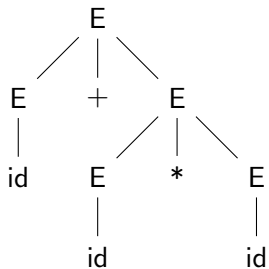
The leaves of a parse tree, read from left to right, constitute a sentential form, called the **yield** or **frontier** of the tree.

Exercise

$$E \rightarrow E + E \mid E * E \mid id$$

Give a derivation of the string $id + id * id$ and draw the parse tree.

Two different Parse Trees



Ambiguous Grammar

A grammar that produces more than one parse tree for some sentence is said to be ambiguous.

Top-Down Parsing

- Construct parse tree, starting from the root and creating nodes in preorder.
- Tree Nodes
 - Nonterminal - choose production, add children
 - Terminal - match with the next input token
- Equivalently finding a Left-most derivation.

Bottom-Up Parsing

- Construct parse tree bottom-up, starting from the leaves.
- Parent node A added to a set of nodes matching the body α of production $A \rightarrow \alpha$.

Further Reading

ALSU: Sections 1.2.2., 2.1, 2.2, 2.5.1, 4.1, 4.2.

Reference

- Aho A. V., Lam M. S., Sethi R., and Ullman J. D., *Compilers: Principles, Techniques, & Tools*, Pearson Education, 2007.